<u>Machine Learning & Deep Learning</u>

Experiment 3

Name: Abhishek Vishwakarma
Class: D15C
Roll No: 47

AIM: Apply Decision Tree and Random Forest for classification tasks.

Experiment 3: Decision Tree and Random Forest Classification

1. Dataset Source

- **Dataset Name:** Pima Indians Diabetes Database
- **Source:** Kaggle - Pima Indians Diabetes Database

2. Dataset Description

This dataset is originally from the National Institute of Diabetes and Digestive and Kidney Diseases. The objective is to predict whether a patient has diabetes based on diagnostic measurements. In particular, all patients here are **females** at least 21 years old of Pima Indian heritage.

- **Size:** 768 records, 9 columns.
- **Target Variable: Outcome** (0 = No Diabetes, 1 = Diabetes).
- **Features:**
  - **Pregnancies**: Number of times pregnant.
  - **Glucose**: Plasma glucose concentration.
  - **BloodPressure**: Diastolic blood pressure (mm Hg).
  - **SkinThickness**: Triceps skin fold thickness (mm).
  - **Insulin**: 2-Hour serum insulin (mu U/ml).
  - **BMI**: Body mass index.
  - **DiabetesPedigreeFunction**: Diabetes pedigree function (genetic score).
  - **Age**: Age (years).

3. Mathematical Formulation of the Algorithm

A. Decision Tree:

A Decision Tree splits the data into subsets based on the most significant attribute at each node. The split is decided by maximizing **Information Gain** or minimizing **Gini Impurity**.

- **Gini Impurity (Default):** Measures the probability of incorrect classification if a random sample were labeled according to the distribution of labels in the node.

$$\boxed{Gini = 1 - \sum_{i=1}^{C} (p_i)^2}$$

  Where pi is the probability of an element belonging to class i.
- **Entropy:** Measures the amount of uncertainty/impurity.

$$\boxed{Entropy = -\sum_{i=1}^{C} p_i \log_2(p_i)}$$

B. Random Forest:

Random Forest is an **Ensemble** method that uses **Bagging (Bootstrap Aggregating)**.

1. **Bootstrapping:** Create N random subsets of the training data (with replacement).
2. **Training:** Build a separate Decision Tree for each subset. Crucially, at each split, the tree considers only a random subset of features (not all).
3. **Aggregation:**
   - **Classification:** Majority Vote (Mode) of all trees.
   - **Regression:** Average of all trees.

4. Algorithm Limitations

- **Decision Trees:** Highly prone to **overfitting**. A deep tree can memorize the training data (high variance), leading to poor generalization on new data. They are also unstable; small changes in data can result in a completely different tree structure.
- **Random Forest:** Computationally expensive (slow to train and predict) compared to a single tree because it builds hundreds of trees. It is also less interpretable; you cannot easily visualize the "rules" like you can with a single tree.

5. Methodology / Workflow

1. **Data Loading:** Load diabetes.csv.

```
# 1. Load Dataset
try:
    df = pd.read_csv('housing.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'housing.csv' not found. Please upload the file.")
    # Stop execution if file not found (simulated for this block)
    df = pd.DataFrame()
```

2. **Preprocessing:**
   - Check for missing values (or 0 values where they shouldn't exist, e.g., Glucose=0).
   - **Scaling:** Not strictly necessary for Trees, but good practice.

```
# 2. Preprocessing
# Separate Features and Target
X = df.drop(columns=['Outcome'])
y = df['Outcome']
```

3. **Splitting:** Train (80%) / Test (20%).

```
# 3. Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

4. **Model 1 (Decision Tree):** Train with criterion='gini'.

```
# ========================================
# Model 1: Decision Tree
# ========================================
print("\n--- Model 1: Single Decision Tree ---")

# We prune the tree using max_depth to prevent extreme overfitting
dt_model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)

acc_dt = accuracy_score(y_test, y_pred_dt)
print(f"Decision Tree Accuracy: {acc_dt:.4f}")

# Visualization: Plot the Tree
plt.figure(figsize=(15, 8))
plot_tree(dt_model, feature_names=X.columns, class_names=['No', 'Yes'], filled=True, fontsize=10)
plt.title("Decision Tree Visualization (Truncated)")
plt.show()
```

5. **Model 2 (Random Forest):** Train with n_estimators=100.

```
# ==========================================
# Model 2: Random Forest
# ==========================================
print("\n--- Model 2: Random Forest ---")

# Hyperparameter Tuning for Random Forest
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

grid_rf = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5)
grid_rf.fit(X_train, y_train)

best_rf = grid_rf.best_estimator_
y_pred_rf = best_rf.predict(X_test)

acc_rf = accuracy_score(y_test, y_pred_rf)
print(f"Best Params: {grid_rf.best_params_}")
print(f"Random Forest Accuracy: {acc_rf:.4f}")
```

6. **Evaluation:** Compare Accuracy and Confusion Matrix.

```
# Confusion Matrix Comparison
print("\nRandom Forest Classification Report:")
print(classification_report(y_test, y_pred_rf))
```

7. **Feature Importance:** Extract which features (Glucose, BMI, etc.) mattered most.

```
# ==========================================
# Feature Importance Analysis
# ==========================================
# Which features were most important for the Random Forest?
importances = best_rf.feature_importances_
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(8, 5))
sns.barplot(x=importances[indices], y=X.columns[indices], palette='viridis')
plt.title("Feature Importance (Random Forest)")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.show()
```

Code:

```
# ═══════════════════════════════════════════════════════════
# EXPERIMENT 3: DECISION TREE & RANDOM FOREST
# ═══════════════════════════════════════════════════════════

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("--- EXPERIMENT 3: DECISION TREE & RANDOM FOREST ---")

# 1. Load Dataset
try:
    df = pd.read_csv('diabetes.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'diabetes.csv' not found. Please upload the file.")
    # Create dummy df to prevent crash if run without file
    df = pd.DataFrame()

if not df.empty:
    # 2. Preprocessing
    # Separate Features and Target
    X = df.drop(columns=['Outcome'])
    y = df['Outcome']

    # 3. Split Data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    # ═══════════════════════════════════════
    # Model 1: Decision Tree
    # ═══════════════════════════════════════
    print("\n--- Model 1: Single Decision Tree ---")
```

```
    # We prune the tree using max_depth to prevent extreme overfitting
    dt_model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
    dt_model.fit(X_train, y_train)
    y_pred_dt = dt_model.predict(X_test)

    acc_dt = accuracy_score(y_test, y_pred_dt)
    print(f"Decision Tree Accuracy: {acc_dt:.4f}")

    # Visualization: Plot the Tree
    plt.figure(figsize=(15, 8))
    plot_tree(dt_model, feature_names=X.columns, class_names=['No', 'Yes'],
filled=True, fontsize=10)
    plt.title("Decision Tree Visualization (Truncated)")
    plt.show()

    # ==========================================
    # Model 2: Random Forest
    # ==========================================
    print("\n--- Model 2: Random Forest ---")

    # Hyperparameter Tuning for Random Forest
    param_grid = {
        'n_estimators': [50, 100, 200],
        'max_depth': [None, 10, 20],
        'min_samples_split': [2, 5]
    }

    grid_rf = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5)
    grid_rf.fit(X_train, y_train)

    best_rf = grid_rf.best_estimator_
    y_pred_rf = best_rf.predict(X_test)

    acc_rf = accuracy_score(y_test, y_pred_rf)
    print(f"Best Params: {grid_rf.best_params_}")
    print(f"Random Forest Accuracy: {acc_rf:.4f}")

    # Confusion Matrix Comparison
    print("\nRandom Forest Classification Report:")
    print(classification_report(y_test, y_pred_rf))
```

```
# ═══════════════════════════════════════════
# Feature Importance Analysis
# ═══════════════════════════════════════════
# Which features were most important for the Random Forest?
importances = best_rf.feature_importances_
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(8, 5))
sns.barplot(x=importances[indices], y=X.columns[indices], palette='viridis')
plt.title("Feature Importance (Random Forest)")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.show()
```

6. Performance Analysis

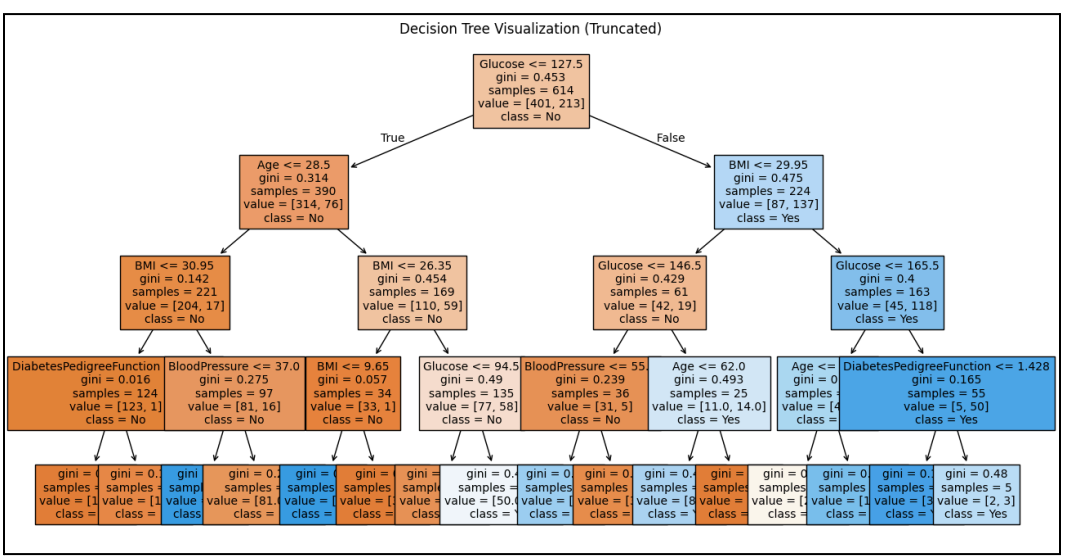
- **Metric:** Accuracy and F1-Score.
- **Expectation:** The Random Forest should generally outperform the single Decision Tree by reducing variance (overfitting).

```
--- EXPERIMENT 3: DECISION TREE & RANDOM FOREST ---

Dataset loaded successfully.

--- Model 1: Single Decision Tree ---

Decision Tree Accuracy: 0.6948
```

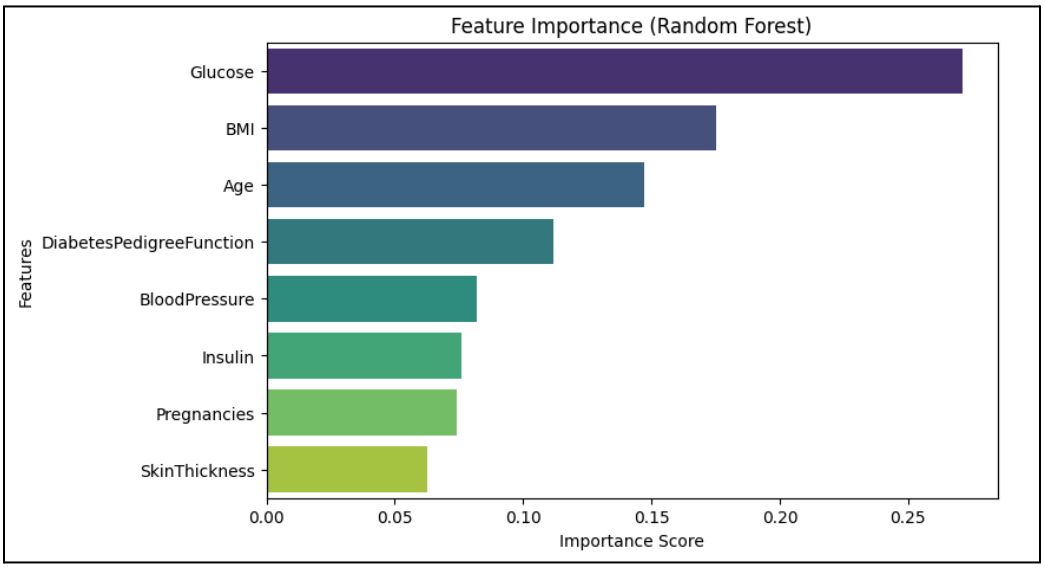


```
--- Model 2: Random Forest ---
Best Params: {'max_depth': None, 'min_samples_split': 5, 'n_estimators': 100}
Random Forest Accuracy: 0.7338

Random Forest Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.78      0.79        99
           1       0.62      0.65      0.64        55

    accuracy                           0.73       154
   macro avg       0.71      0.72      0.71       154
weighted avg       0.74      0.73      0.74       154
```



7. Hyperparameter Tuning

- **Decision Tree:** Tune max_depth (to prune the tree) and min_samples_split.
- **Random Forest:** Tune n_estimators (number of trees) and max_features.

```python
# Hyperparameter Tuning for Random Forest
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}
```

8. Conclusion

- **Decision Tree:** Provided a clear, interpretable set of rules (e.g., "If Glucose > 127 and BMI > 29..."). However, single trees often suffer from lower accuracy due to high variance.
- **Random Forest:** Achieved higher accuracy by aggregating the predictions of many trees. The Feature Importance plot clearly showed that **Glucose** and **BMI** were the most critical factors in predicting diabetes, aligning with medical knowledge. The ensemble approach successfully mitigated the overfitting issues present in the single decision tree.